

COURSE MATERIAL



Programme Overview & Banking Data Model

Programme Overview

Programme Overview & Banking Data Model

How to use this programme kit

This kit contains everything needed to deliver **Enterprise Data Engineering with SQL, Redis & Kafka** - a 120-hour instructor-led programme - to a batch of fresh graduates.

Every module has its own folder. Inside each folder you will find five deliverables.

File	Who uses it	When
Mnn_Presentation.pptx	Trainer	During the classroom session
Mnn_Course_Material.pdf	Learner	Read-along reference, before and after class
Mnn_Lab_Guide_with_Solutions.pdf	Trainer + learner	Guided hands-on, worked step by step
Mnn_Exercises.pdf	Learner	Independent practice, no answers shown
Mnn_Exercise_Solutions.pdf	Trainer	Released after the practice window closes

Distribute the exercises without the solutions. The two are deliberately separate files so a trainer can hand out `Exercises.pdf` in the lab and release `Exercise_Solutions.pdf` at the end of the day.

The one rule that holds the programme together

Every example, lab, exercise and solution in all twelve modules uses **one bank, one schema, one dataset**: ICICI Bank. A learner never has to re-learn a new toy database. The `customer` table introduced in Module 2 is the same `customer` table that gets normalized in Module 4, indexed in Module 5, cached in Redis in Module 7, streamed through Kafka in Module 8, migrated in Module 11, and demonstrated in the Module 12 capstone.

About the data. Every customer, account, transaction, loan and employee in this kit is **synthetic**, generated for teaching. Nothing here is drawn from any real system, and no real customer data appears anywhere in the programme. The branch names, IFSC codes, account numbers and balances are all invented. The schema is a simplified teaching model of a core banking system, not a description of any bank's actual production design.

The ICICI Bank story

ICICI Bank is a mid-sized Indian retail bank with 8 branches, roughly 30,000 customers and about 2 million transactions a month. The bank runs a classic core banking system on a relational database, and it is in the middle of a modernization programme.

The learner joins as a data engineer on that modernization team. Each module is a work item on that team's backlog.

Module	The business problem the learner is handed
1	The core banking database is a black box. Understand what an RDBMS actually is and how ICICI Bank's data is organised.
2	Build the customer and account tables, and load the first branch's data correctly.
3	Branch heads want daily reports: top customers, channel-wise volumes, dormant accounts.
4	The customer table has duplicated branch and address data. Redesign it properly.
5	The mini-statement screen takes 9 seconds to load. Make it fast, and move the logic into stored procedures.
6	A fund transfer debited one account and never credited the other. Fix it with proper transactions.
7	The balance-enquiry API hammers the database 4,000 times a minute. Put Redis in front of it.
8	Fraud detection, SMS alerts and the ledger all need the same transaction event. Publish it once on Kafka.
9	An audit found shared database logins and no tested restore. Lock it down and prove recovery works.
10	The bank's reporting platform runs on Microsoft SQL Server. Administer and develop on it.
11	The legacy Oracle core is being retired. Migrate schema and data to SQL Server and PostgreSQL.
12	Build the whole thing end to end and present it.

The core banking data model

Nine tables. Learn them once, use them for 120 hours.

Table	What it holds	Key columns
branch	ICICI Bank's physical branches	branch_id, branch_name, ifsc_code, city, state, opened_date
customer	Everyone who banks with ICICI	customer_id, first_name, last_name, date_of_birth, email, phone, city, kyc_status, risk_rating, customer_since, home_branch_id
account	Savings, current, salary and NRI accounts	account_id, account_number, customer_id, branch_id, account_type, balance, currency, status, opened_date, interest_rate
account_txn	Every debit and credit posted	txn_id, account_id, txn_date, txn_type, amount, channel, narration, status, reference_no
loan	Home, auto, personal, education, gold loans	loan_id, customer_id, branch_id, loan_type, principal_amount, interest_rate, tenure_months, emi_amount, disbursed_date, outstanding_amount, status
loan_payment	The EMI schedule and what was actually paid	payment_id, loan_id, due_date, paid_date, amount_due, amount_paid, status
card	Debit and credit cards on an account	card_id, account_id, card_type, card_number_masked, issued_date, expiry_date, status, daily_limit
beneficiary	Payees a customer registered for transfers	beneficiary_id, customer_id, beneficiary_name, beneficiary_acct, ifsc_code, added_on, is_active
employee	Branch staff, with a manager hierarchy	employee_id, first_name, last_name, branch_id, designation, manager_id, hire_date, salary

How the tables relate

```

branch (1) ----< customer (1) ----< account (1) ----< account_txn
|                                     |
|                                     +----< card
|                                     +----< loan (1) ----< loan_payment
|                                     +----< beneficiary
+----< employee --+
      ^           |
      +-----+ manager_id is a self-referencing FK
  
```

Read ----< as "one to many". One branch has many customers; one customer has many accounts; one account has many transactions.

Why `account_txn` and not `transaction`? `TRANSACTION` is a reserved word in most SQL dialects. Naming the table `account_txn` avoids quoting it everywhere, and it is a habit worth building early.

Setting up the classroom environment

What you need installed

Software	Used from	Notes
PostgreSQL 18	Module 1	Primary teaching database. Free, fast to install, good error messages.
Oracle XE 21c	Module 1	Needed for Oracle-specific syntax and for the Module 11 migration source.
MySQL 8	Module 1	Third dialect for comparison.
pgAdmin / DBeaver	Module 1	A visual client makes execution plans much easier to teach.
Redis 7	Module 7	Docker is the fastest route on Windows.
Apache Kafka 3.x	Module 8	Runs in KRaft mode, no ZooKeeper needed.
Microsoft SQL Server 2022 Developer + SSMS	Module 10	Developer edition is free and full-featured.
SSMA for Oracle	Module 11	Free download from Microsoft.
Python 3.11+ or Java 17	Module 7	For the Redis and Kafka client code.

Loading the ICICI Bank database

The `sql` folder beside this document contains the schema in four dialects and the seed data in two.

Run these at the **PowerShell** prompt, not inside `psql`. Each one connects, does its work and disconnects, so each asks for the password separately.

```
# PostgreSQL
createdb -U postgres icicibank
psql -U postgres -d icicibank -f sql/01_schema_postgres.sql
psql -U postgres -d icicibank -f sql/02_seed_postgres.sql
```

Fix the PATH first. The PostgreSQL installer does not add `C:\Program Files\PostgreSQL\18\bin` to the Windows PATH, and every command line tool lives there. Until it is added and a **new** window opened, `psql` and `createdb` are both "not recognized". Verify with `Get-Command psql`, `createdb`. Full details are in [M00_Student_Setup_Guide.pdf](#).

```
-- Microsoft SQL Server (from SSMS or sqlcmd)
CREATE DATABASE ICICIBank;
GO
USE ICICIBank;
```

```
GO
:r sql\01_schema_sqlserver.sql
:r sql\02_seed_sqlserver.sql
```

For Oracle and MySQL, run `01_schema_oracle.sql` or `01_schema_mysql.sql` and then the PostgreSQL seed file, which uses ANSI `DATE '...'` and `TIMESTAMP '...'` literals that both engines accept.

Confirming the load worked

Run this after seeding. Every learner in the room must see identical numbers, because the seed data is generated deterministically.

```
SELECT 'branch' AS table_name, COUNT(*) AS row_count FROM branch
UNION ALL SELECT 'customer',      COUNT(*) FROM customer
UNION ALL SELECT 'account',       COUNT(*) FROM account
UNION ALL SELECT 'account_txn',   COUNT(*) FROM account_txn
UNION ALL SELECT 'loan',          COUNT(*) FROM loan
UNION ALL SELECT 'loan_payment',  COUNT(*) FROM loan_payment
UNION ALL SELECT 'card',          COUNT(*) FROM card
UNION ALL SELECT 'beneficiary',   COUNT(*) FROM beneficiary
UNION ALL SELECT 'employee',      COUNT(*) FROM employee
ORDER BY table_name;
```

Expected: 8 branches, 30 customers, 36 accounts, 495 transactions, 24 employees, 12 loans and 144 loan payments. If a learner's counts differ, they ran the seed file twice - drop and recreate.

Reset between modules. Modules 2, 4 and 11 deliberately modify the schema. Re-running `01_schema_*.sql` followed by `02_seed_*.sql` restores a clean bank in under a minute. Tell learners this on day one so nobody is afraid to experiment.

Suggested 120-hour schedule

Day	Hours	Modules covered
1	6	Module 1 - RDBMS fundamentals
2	8	Module 2 - SQL fundamentals
3	8	Module 3 - Advanced SQL queries
4	8	Module 4 - Design and normalization
5	8	Module 5 - Optimization and programming
6	6	Module 6 - Transactions and advanced SQL
7-8	10	Module 7 - NoSQL and Redis
9-10	10	Module 8 - Apache Kafka
11	6	Module 9 - Security and integration
12-14	20	Module 10 - SQL Server administration
15-17	20	Module 11 - Enterprise data migration
18-19	10	Module 12 - Capstone project

Running a module session

A module hour splits roughly 45 / 15 into teaching and doing.

1. **Open with the business problem.** Every module deck starts with the ICICI Bank ticket that motivates the topic. Spend five minutes there; fresh graduates retain syntax far better when it solves a problem they can picture.
2. **Teach from the deck**, stopping at each code slide to run the query live against the seeded database.
3. **Run the lab.** The lab guide is written to be followed keystroke by keystroke, with the expected output shown after each step.
4. **Set the exercises.** Hand out only [Exercises.pdf](#). The exercises are graded Level 1 to Level 3, so a fast learner always has somewhere to go.
5. **Debrief with the solutions.** The solution file explains why, not just what, and flags the mistakes that batches actually make.